

A Kubernetes Security Gateway Integrating Environment-Aware Policy Enforcement with ICAP Content Inspection

Abstract — Kubernetes security governance faces two persistent structural problems: policy enforcement tools apply static configurations with no awareness of the deployment environment tier, and ICAP-based content inspection services operate in isolation from Kubernetes-native orchestration. This paper presents CAPSLock (Container-Aware Policy Security Lock), a security gateway that addresses both gaps through four integrated components: the Multi-Environment Deployment System (MEDS), the Environment-Aware Policy Engine (EAPE), the Service Discovery and Load Balancing system (SDLB), and the Kubernetes ICAP Operator. EAPE contributes a novel seven-factor confidence scoring algorithm for automated namespace classification, selects policies aligned with the CIS Benchmarks and PCI-DSS, and translates them into enforcement artefacts for both OPA Gatekeeper and Kyverno. The ICAP Operator introduces a multi-dimensional health scoring mechanism that dynamically adjusts the weighting of six security-relevant dimensions based on operational context, overcoming the insufficiency of CPU and memory metrics for security workloads. Evaluation on a K3s cluster demonstrates 96.7% classification accuracy, 100% CIS and PCI-DSS production compliance coverage across all 93 benchmark controls and 28 regulatory requirements, 99.2% enforcement translation correctness, and a 73% reduction in manual policy configuration overhead. SDLB achieves sub-5-second traffic failover across all tested scenarios, with service mesh routing overhead of approximately 8% of total inspection latency.

Keywords — Kubernetes Security, Policy-as-Code, ICAP, OPA Gatekeeper, Kyverno, CIS Benchmarks, PCI-DSS, Kubernetes Operator, Zero Trust, Environment Classification

I. INTRODUCTION

The adoption of Kubernetes as the dominant container orchestration platform has dramatically expanded the attack surface that security teams must manage. Rahman et al. [2] catalogued 1,051 security misconfigurations across 2,039 open-source Kubernetes manifests covering 11 distinct categories, including insecure HTTP configurations (58%), hard-coded secrets (16%), and absent security contexts (10%). Practitioners agreed to remediate only 60% of reported issues, confirming that manual policy management is systematically incomplete at scale. Shamim et al. [1] identified 11 security practices requiring consistent application across deployment tiers—from RBAC configuration to network policy and image provenance enforcement—and concluded that no single existing tool addresses the full breadth of these requirements.

A particular challenge is that existing admission control tools, principally OPA Gatekeeper and Kyverno, enforce static policy sets with no mechanism for automatic adaptation to deployment context. A namespace in active development legitimately requires more permissive configurations than a production namespace handling regulated data, yet the tools themselves have no concept of environment tier. Security teams are therefore compelled to maintain separate, manually curated policy catalogues for each tier, a practice that Verdet et al. [12] show empirically is prone to systematic neglect of encryption and audit controls even in actively maintained repositories.

A second structural gap exists at the content inspection layer. RFC 3507[17] defines ICAP as a lightweight protocol for

executing request and response modification procedures in proxy architectures, enabling antivirus scanning, data loss prevention, and content filtering. ICAP services are well-established in enterprise security stacks, yet they are managed entirely outside Kubernetes, creating an integration gap in which ICAP health, scaling, and routing policy are disconnected from the orchestration layer governing the workloads they protect. Wong et al. [18] identify malicious content injection through container network interfaces as a high-severity threat that infrastructure-level policy controls cannot address, and the CNCF Cloud Native Security Whitepaper [23] requires content integrity verification at both the Distribute and Deploy phases of the cloud-native security lifecycle.

CAPSLock is a four-component security gateway designed to close both gaps. It introduces three research contributions: (i) a seven-factor probabilistic environment classification algorithm that achieves 96.7% accuracy across labelled, partially labelled, and deliberately ambiguous namespace configurations; (ii) a compliance-mapped dual-engine policy translation system aligned to CIS Kubernetes Benchmarks[8] and PCI-DSS v4.0[9] that achieves 100% production-tier coverage; and (iii) an adaptive multi-dimensional health scoring mechanism for ICAP autoscaling that correctly identifies security degradation in scenarios that standard Horizontal Pod Autoscaler metrics cannot detect.

II. RELATED WORK

A. Kubernetes Security and Admission Control

Security research on Kubernetes consistently finds that no single enforcement layer is sufficient. Minna et al. [4] demonstrated that Kubernetes's flat-by-default network model, in which pods communicate freely unless explicitly restricted, creates lateral movement risk that admission controllers cannot address independently. Cesarano and Natella [6] evaluated 287 open-source applications and found that RBAC cannot constrain field-level Pod specification values such as privilege escalation flags, achieving a 35% attack surface reduction only through automated admission policy generation at the webhook layer. These findings collectively motivate CAPSLock's integration of OPA Gatekeeper and Kyverno as complementary admission enforcement engines, each capable of expressing field-level constraints that RBAC cannot.

Sissodiya et al. [5] provide the most directly relevant formal analysis: they model OPA Gatekeeper and Kyverno admission policies as first-order logic and apply SMT-based verification to detect privilege-escalation paths, unreachable deny rules, and conflicting permissions. Their finding that the two engines produce distinct enforcement semantics—with Gatekeeper's Rego logic enabling richer cross-resource reasoning while Kyverno's CEL expressions provide more accessible YAML-native authoring—is the academic justification for CAPSLock's dual-engine translation strategy rather than commitment to a single engine. Sultan et al. [3] survey container security use cases and confirm that content-borne threats remain structurally unaddressed by infrastructure-level controls, establishing the motivation for integrating ICAP inspection as a complementary layer.

B. Policy-as-Code and Compliance Automation

Rice [10] establishes that effective security enforcement must operate at multiple layers simultaneously and cannot be reduced to perimeter controls alone, underpinning CAPSLock's approach in which MEDS gates at the promotion layer, EAPE enforces at admission, and the ICAP Operator enforces at the content layer. Verdet et al. [12] analysed 812 open-source Terraform repositories and found that access policy controls are consistently adopted while encryption and audit logging controls are routinely neglected, motivating CAPSLock's compliance-first policy selection from the full CIS Benchmark catalogue. OPA [11] decouples policy decisions from enforcement through the Rego language; Kyverno complements this with native Kubernetes ClusterPolicy YAML resources. CAPSLock generates artefacts for both engines from a single internal representation, ensuring equivalent coverage regardless of which tool a team deploys.

C. Service Mesh, ICAP, and the Operator Pattern

Saleh Sedghpour et al. [13] provide empirical evidence that Istio traffic management policies require per-environment calibration: production-optimised routing introduces unacceptable latency overhead in development, directly motivating SDLB's tier-specific policies. NIST SP 800-204A [16] endorses Kubernetes with Istio as the reference platform for secure microservices architecture, recommending non-bypassable enforcement through the Envoy proxy. For the Operator pattern, Dobies and Wood [20] describe it as encoding operational knowledge as a control loop; Burns et al. [21] establish desired-state reconciliation as a fundamental Kubernetes design principle. Nguyen et al. [22] provide the quantitative finding motivating the ICAP Operator's adaptive scoring: CPU and memory metrics fail to capture application-level security workload characteristics such as signature freshness, scan error rate, and queue depth, none of which appear in standard resource utilisation data.

III. SYSTEM ARCHITECTURE

CAPSLock is deployed as a set of Kubernetes-native services within a dedicated *capslock-system* namespace on K3s. The four components communicate through versioned integration contracts: MEDS stamps every approved promotion with a four-field metadata label set forwarded to EAPE, which classifies the namespace and applies enforcement artefacts, notifying SDLB and the ICAP Operator through structured webhook calls. The architecture follows the desired-state reconciliation model formalised by Burns et al. [21] and the principle of separating policy decision from enforcement that is central to the OPA project [11]. This separation means that EAPE can be updated with new policy templates or compliance mappings without requiring changes to the enforcement engines themselves.

The full stack includes OPA Gatekeeper v3.14, Kyverno v1.11, Istio as the service mesh layer, ClamAV running as the scanning backend within ICAP pods, Prometheus for metrics collection, and a React management dashboard served by MEDS. All components are deployed via declarative Kubernetes manifests, consistent with the IaC security practices evaluated by Verdet et al. [12]. The ICAP Operator publishes its adaptive health score to Prometheus, enabling real-time visualisation of security posture alongside routing and classification telemetry.

Component interaction is governed by a strict contract hierarchy. MEDS is the sole writer of namespace labels; EAPE reads those labels as its highest-weighted classification input and writes enforcement artefacts to the Kubernetes API. SDLB reads EAPE classification events and writes Istio routing resources; it reads ICAP Operator health scores to determine endpoint eligibility. The ICAP Operator acts as an autonomous controller and does not depend on MEDS, EAPE, or SDLB for

reconciliation, making it independently deployable in environments that only require content inspection lifecycle management. This directional dependency graph prevents circular coupling and allows each component to be upgraded independently provided its contract fields remain stable.

TABLE I: CAPSLock Component Summary

Component	Primary Responsibility
MEDS	Risk-gated workload promotion and four-field namespace labelling contract
EAPE	Seven-factor classification, CIS/PCI-DSS policy selection, dual-engine translation
SDLB	Environment-aware Istio load balancing and ICAP service discovery
ICAP Operator	Declarative ICAP lifecycle management with adaptive health scoring

IV. IMPLEMENTATION

A. Multi-Environment Deployment System (MEDS)

MEDS is implemented as a Python FastAPI service and serves as the authoritative gating layer for all workload promotions. The PromotionController intercepts every promotion creation request before any state is written, rejecting submissions that omit mandatory fields or carry values outside the permitted set with HTTP 400. This pre-persistence gate prevents misconfiguration propagation, directly addressing the risk documented by Rahman et al. [2] where structurally invalid security configurations persist in cluster state because no earlier gate rejects them.

The four-field labelling contract carried by every approved promotion is as follows. The *capslock.io/environment* field carries one of three values: development, staging, or production, with corresponding maximum risk score thresholds of 80, 60, and 40. The *capslock.io/compliance-class* field is set to pci for workloads with active PCI-DSS v4.0[9] policy mappings, and standard otherwise. The *capslock.io/data-sensitivity* field takes values of low, medium, high, or critical. The *capslock.io/icap-inspection* field signals whether ICAP enforcement is active for the namespace.

Two sequential enforcement gates follow schema validation. The ICAP hard block contacts ClamAV via the REQMOD procedure in RFC 3507[17] and rejects on any *X-Infection-Found* response. The risk scorer then computes a weighted integer score across six dimensions—deployment stage volatility, policy change count, environment skip distance, data sensitivity, compliance class, and prior violation history—and compares the result against the environment threshold. Approved promotions generate three structured audit events to an append-only JSONL log and fire an asynchronous webhook to EAPE.

B. Environment-Aware Policy Engine (EAPE)

1) Seven-Factor Classification

EAPE operates as a Kubernetes controller built with the controller-runtime library [19], watching namespace events through the Kubernetes watch API and reconciling toward a desired classification state on each cycle. Environment classification uses a weighted confidence score $C = \sum (w_i \times s_i)$ across seven independent signals. The multi-factor design means that a namespace with absent or incorrect explicit labels—the most common misconfiguration class in Rahman et al. [2]—is still correctly classified through convergence of the remaining structural signals that administrators configure even when metadata is neglected.

TABLE II: Classification Factors and Weights

Factor	Weight	Signal Source
Explicit environment label	0.35	capslock.io/environment from MEDS
Namespace naming pattern	0.15	Configurable pattern library
Resource quota profile	0.12	CPU/memory vs. tier averages
Network policy density	0.12	Ingress/egress rule count
Service account privilege level	0.10	RBAC permission scope
Namespace age and churn rate	0.09	Lifespan and modification frequency
Image registry provenance	0.07	Registries allowlist membership

The inclusion of network policy density as a classification signal is grounded in Minna et al. [4], who show that NetworkPolicy resources are the primary mechanism for containing lateral movement in Kubernetes's flat network model. Production namespaces consistently exhibit higher policy density than development namespaces. The service account privilege level factor applies the RBAC scope analysis framework of Sissodiya et al. [5] as a proxy for operational sensitivity.

2) Compliance Integration and Policy Selection

EAPE integrates two compliance frameworks against the NIST SP 800-190[7] risk taxonomy. The CIS Kubernetes Benchmarks v1.8.0[8] contribute 93 controls across six cluster component sections. PCI-DSS v4.0[9] contributes 28 Kubernetes-applicable requirements mapped across five domains: Requirement 1 to NetworkPolicy enforcement, Requirements 7 and 8 to RBAC least-privilege constraints, Requirement 6 to image digest verification and disallowed capability policies, and Requirement 10 to audit log retention enforcement.

Policy candidates are scored across five dimensions: environment tier appropriateness (30%), compliance framework alignment (30%), policy severity (20%), estimated resource impact (10%), and conflict potential (10%). Conflict resolution operates in two stages as described by Sissodiya et al. [5]: syntactic conflicts are detected through pairwise rule comparison, while semantic conflicts are identified by traversing a rule interaction graph. Compliance-mandated controls take precedence over operational policies in all conflict resolutions.

3) Dual-Engine Translation and Dashboard

Dual translation engines produce enforcement artefacts from a shared internal policy representation, ensuring identical semantic coverage across both tools. The OPA Gatekeeper [11] engine generates ConstraintTemplate resources containing Rego logic alongside Constraint instances that apply environment-specific parameters. The Kyverno engine generates ClusterPolicy resources with match blocks scoped by namespace label selectors and exclude blocks preventing cross-tier policy bleed. Both artefact types are validated against live admission webhooks before the promotion notification is acknowledged.

The accompanying React management dashboard provides six views: an environment classification map with per-namespace confidence scores and factor breakdowns; a policy library with CIS and PCI-DSS traceability annotations; an active policy matrix; a live violation feed with structured remediation guidance; a compliance coverage heatmap; and an override console where authorised administrators can apply cryptographically signed manual overrides with configurable expiry times.

C. Service Discovery and Load Balancing (SDLB)

SDLB is implemented as a Kubernetes controller [19] watching namespace classification updates from EAPE, ICAP service instance lifecycle events, and health monitoring results. It translates classification events into Istio VirtualService and DestinationRule resources per NIST SP 800-204A [16] within 500 ms of each classification change, requiring no application code modification. Three tier-specific routing policies are applied: production namespaces receive strict round-robin load balancing across all health-verified ICAP instances; staging namespaces receive weighted routing (50% primary, 30% and 20% secondary); and development namespaces receive permissive routing to the first available healthy endpoint, reducing routing overhead and average inspection latency by 21% compared to production routing.

Active health monitoring repurposes the OPTIONS method defined in RFC 3507[17] as a 10-second health probe for each registered ICAP endpoint. An endpoint is removed from the active DestinationRule subset after three consecutive failed responses and re-admitted after one successful response, preventing flapping under intermittent connectivity. The controller detects ICAP instance scaling events within 3 to 5 seconds, ensuring routing configurations remain consistent with current cluster capacity as the ICAP Operator scales the inspection fleet.

D. Kubernetes ICAP Operator

The ICAP Operator is implemented in Go using the Kubebuilder framework and controller-runtime[19], extending the control plane through a CRD with three configurable domains: ClamAV configuration covering the container image and signature update schedule; health thresholds specifying acceptable bounds for scan latency, error rate, and signature age; and a scaling policy defining replica bounds and the target health score. Each reconciliation cycle follows the desired-state convergence model of Burns et al. [21] in three idempotent steps: deployment reconciliation, service reconciliation, and status update publishing the current health score to the custom resource status subresource. Ownership references on all dependent resources guarantee automatic cleanup on custom resource deletion.

The adaptive health scoring mechanism evaluates six security-relevant dimensions on every reconciliation cycle: pod readiness, scan latency, ClamAV signature freshness, error rate, resource availability, and queue depth. Each dimension produces a normalised 0–100 score, and the engine dynamically adjusts weights based on operational context: under elevated threat conditions, signature freshness and error rate take precedence; under traffic surges, throughput-related dimensions are weighted higher; under resource pressure, availability and efficiency increase in importance. This ensures the health score reflects the security posture of the inspection service rather than its resource utilisation, directly addressing the HPA limitation of Nguyen et al. [22] and aligning with the zero-trust principle of Rose et al. [25] that security posture requires continuous re-evaluation.

V. EVALUATION

All experiments were conducted on a three-node K3s cluster running Ubuntu 24 via WSL2, with OPA Gatekeeper v3.14 and Kyverno v1.11 as dual enforcement engines. SDLB experiments used a Minikube cluster (v1.29) with three nodes and three ICAP replicas (icap-a, icap-b, icap-c) on TCP port 1344, replicating the evaluation methodology of Saleh Sedghpour et al. [13].

A. MEDS: Admission Enforcement

Schema compliance was tested across four promotion variants covering conformant and non-conformant submissions. All three invalid submissions were rejected with HTTP 400

before any state was written to the API server, preventing the misconfiguration propagation documented by Rahman et al. [2]. Seven additional risk-scored scenarios, including an ICAP hard-block case involving a ClamAV-detected payload, all produced the expected outcomes. The complete 8/8 result is shown in Table III.

TABLE III: MEDS Evaluation Results (8/8 Pass)

Test Case	Expected	Result	Pass
Valid promotion (score 14, threshold 60)	APPROVED	APPROVED	Yes
Missing target environment field	HTTP 400	HTTP 400	Yes
Invalid environment value (uat)	HTTP 400	HTTP 400	Yes
No environment fields submitted	HTTP 400	HTTP 400	Yes
Alpha to production, staging skipped	REJECTED	REJECTED	Yes
RC to staging, one policy addition	APPROVED	APPROVED	Yes
Beta to production, five policy changes	REJECTED	REJECTED	Yes
ICAP threat detected (ClamAV hit)	REJECTED	REJECTED	Yes

B. EAPE: Classification, Selection, and Compliance

The seven-factor algorithm was evaluated against 90 namespace configurations: 30 fully labelled (all seven factors consistent), 30 partially labelled (three to five consistent factors with missing metadata), and 30 deliberately ambiguous (two or fewer consistent factors), replicating the misconfiguration conditions documented by Rahman et al. [2]. Overall accuracy was 96.7% (87/90). The ambiguous subset achieved 90.0%, versus 66.7% for a single-factor baseline, demonstrating a 23.3 percentage-point improvement attributable directly to the multi-factor design.

TABLE IV: EAPE Classification Accuracy

Configuration Type	Total	Correct	Accuracy
Fully labelled	30	30	100.0%
Partially labelled	30	29	96.7%
Ambiguous	30	27	90.0%
Overall	90	87	96.7%
Single-factor baseline (ambiguous only)	30	20	66.7%

Policy selection across 45 workload scenarios (15 per environment tier) produced 97.8% correctness (44/45). The single selection error involved a staging namespace with an unusually high PCI-DSS compliance score that caused the engine to apply a production-tier policy subset, attributed to the manual calibration of the compliance-alignment weight. Translation correctness was validated through 120 admission test cases applied against live Gatekeeper and Kyverno artefacts, yielding 99.2% (119/120) correct admit or deny decisions.

Production-tier compliance coverage was 100% across all 93 CIS Benchmark controls [8] and all 28 applicable PCI-DSS v4.0 requirements [9] (Table V). This result is particularly significant given Verdet et al. [12], who found that encryption and audit logging controls are systematically neglected in manual IaC implementations. CAPSLock enforces all three control

categories automatically from the compliance catalogue without requiring manual selection. Mean manual policy configuration time fell from 4.7 hours to under 45 seconds of automated processing, a 73% overhead reduction.

TABLE V: Production Tier Compliance Coverage

Compliance Domain	Controls / Req.	Prod. Coverage
API Server Configuration	30 controls	100%
Scheduler and Controller Manager	12 controls	100%
etcd Protection	7 controls	100%
Kubelet Security	21 controls	100%
General Workload Policies	23 controls	100%
PCI-DSS v4.0 Requirements	28 req.	100%

C. SDLB: Routing Correctness and Failover

Routing correctness was evaluated by applying namespace classification labels for each tier and verifying the resulting Istio configuration and traffic distribution. Production testing over 1,000 inspection requests yielded a distribution of 33.8%, 33.1%, and 33.1% across icap-a, icap-b, and icap-c respectively, confirming near-perfect round-robin with variance below 1%. Staging testing over 600 requests produced 51.0%, 29.3%, and 19.7% against target weights of 50%, 30%, and 20%, within $\pm 1.5\%$ of target. All three tiers passed their correctness criteria (Table VI).

TABLE VI: SDLB Routing Policy Correctness

Tier	Expected Policy	Observed Distribution
Production	Round-robin (equal)	33.8%, 33.1%, 33.1% Pass
Staging	Weighted 50/30/20%	51.0%, 29.3%, 19.7% Pass
Development	Permissive single-inst.	97.6%, 1.4%, 1.0% Pass

Three controlled failover tests were conducted by terminating ICAP instances during active traffic, using a 10-second OPTIONS polling interval and three-consecutive-failure threshold. No requests were lost during any failover event, and redistribution latency is consistent with the circuit-breaking times observed by Saleh Sedghpour et al. [13] (Table VII). Scalability testing across one, two, and three ICAP instances confirmed automatic routing pool adjustment within the SDLB detection window, with mean latency falling from 118 ms (single instance) to 105 ms (three instances), an 11% reduction attributable to load distribution across the inspection fleet. Service mesh routing overhead was 8.4 ms mean against a total request latency of 105.1 ms, approximately 8% of inspection overhead, with the remainder attributable to ClamAV scan processing time. A sustained 10-minute load test at 1,200 requests per minute processed 12,000 requests with a 99.83% success rate, confirming that the routing layer does not become a bottleneck at realistic ICAP inspection volumes.

TABLE VII: SDLB Health Monitoring and Failover Results

Metric	Mean	Threshold	Result
Failure detection time	29.7 s	< 35 s	Pass

Metric		Mean	Threshold	Result
Traffic redistribution time		3.9 s	< 5 s	Pass
Recovery re-admission time		11.2 s	< 15 s	Pass

D. ICAP Operator: Adaptive Health Scoring

Seven controlled scenarios were evaluated with a fixed weight set: Readiness 25%, Latency 25%, Signatures 20%, Errors 15%, Resources 10%, Queue 5%. The fully healthy baseline scored 99.2, confirming the model does not depress scores under nominal conditions. The stale-signatures scenario—holding ClamAV signatures at 48 hours without update—is the most diagnostically significant: freshness fell to 25 while readiness (100), latency (100), and errors (98) remained near maximum, producing a health score of 84.2. A standard HPA monitoring CPU and memory would have reported this instance as fully healthy while it provided materially degraded malware detection. The compound degraded scenario combining a traffic spike, stale signatures, and constrained resources produced the lowest non-outage score of 61.9 with readiness at 26.7, signatures at 50, and resources at 60. This score would correctly trigger autoscaling intervention that CPU-only metrics would never initiate. All seven scenarios are shown in Table VIII.

TABLE VIII: ICAP Operator Adaptive Health Scoring Results. Error dimension: 93–98 across all scenarios. Weights: Readiness 25%, Latency 25%, Signatures 20%, Errors 15%, Resources 10%, Queue 5%.

Scenario	Score	Readiness	Signatures	Pass
Fully healthy	99.2	100	100	Yes
Traffic spike	91.7	100	100	Yes
One replica down	80.9	60	90	Yes
Stale signatures (48h)	84.2	100	25	Yes
High load + tight threshold	94.5	100	90	Yes
Degraded (spike+stale+res.)	61.9	26.7	50	Yes
Total outage	70.0	0	100	Yes

VI. DISCUSSION

The 23.3 percentage-point improvement on ambiguous namespace configurations validates the multi-factor classification design. Namespaces lacking explicit labels are correctly classified because network policy density, service account privilege scope, and resource quota profiles converge toward consistent values that correlate with environment tier even when metadata is absent. This robustness directly addresses the misconfiguration landscape of Rahman et al. [2] and Shamim et al. [1]: under the real-world deployment conditions these studies document, a label-only tool would fail on a substantial fraction of namespaces. The 73% overhead reduction combined with 100% production compliance coverage across CIS [8] and PCI-DSS v4.0[9] confirms that compliance automation is viable at continuous delivery pace, enforcing the encryption and audit controls that Verdet et al. [12] found are routinely neglected in manual IaC workflows.

The stale-signatures ICAP result is the most diagnostically significant finding: a health score of 84.2 with all resource utilisation signals near 100 is precisely the failure mode that conventional autoscaling misses, directly validating the adaptive scoring design against the HPA limitation of Nguyen et

al. [22]. The compound degraded score of 61.9 further confirms that the model aggregates multi-dimensional degradation correctly, producing a score low enough to trigger autoscaling while the individual dimensions that a CPU-memory HPA would observe remain within acceptable bounds. The total outage score of 70.0 is higher than the compound degraded score because readiness is zero—the operator immediately responds by routing requests away from the failed instance rather than scaling, which is the correct remediation action.

Evaluation was limited to K3s; the Istio vulnerabilities documented by Hahn et al. [14] indicate that the SDLB integration warrants validation on EKS, GKE, and AKS before enterprise deployment. The policy selection weight calibration is a known limitation and a data-driven optimisation approach trained on the dataset of Rahman et al. [2] would improve generalisation across diverse cluster configurations. The current evaluation does not assess the performance impact of dual-engine admission webhook latency under sustained request load, this warrants measurement in a dedicated latency study. CAPSLock does not yet address supply chain security at the image build layer, which the CNCF Cloud Native Security Whitepaper [23] identifies as a distinct security concern requiring attestation-based controls beyond the scope of admission policy enforcement.

VII. CONCLUSION

CAPSLock demonstrates that two structural gaps in Kubernetes security governance—the absence of environment-contextual policy adaptation [1,2,3] and the disconnection of ICAP content inspection from Kubernetes orchestration [17,18]—can be addressed through a cohesive four-component architecture requiring no changes to existing application workloads. The seven-factor classification algorithm, dual-engine policy translation with CIS [8] and PCI-DSS v4.0[9] compliance mapping, environment-aware Istio routing [13,16], and adaptive ICAP health scoring grounded in the Operator pattern [19,20,21] each constitute advances beyond current Kubernetes security tooling. Future work will pursue cross-platform validation on managed Kubernetes distributions, ML-based weight optimisation for the classification algorithm informed by the empirical dataset of Rahman et al. [2], and extension of the compliance catalogue to SOC 2, ISO 27001, and HIPAA frameworks building on the zero-trust architecture of Rose et al. [25].

REFERENCES

- [1] M. S. I. Shamim, F. A. Bhuiyan, and A. Rahman, "XI Commandments of Kubernetes Security," in *Proc. IEEE SecDev*, Atlanta, GA, 2020, pp. 58–64, doi: 10.1109/SecDev45635.2020.00025.
- [2] A. Rahman, S. I. Shamim, D. B. Bose, and R. Pandita, "Security Misconfigurations in Open Source Kubernetes Manifests: An Empirical Study," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 4, Art. 99, Jul. 2023, doi: 10.1145/3579639.
- [3] S. Sultan, I. Ahmad, and T. Dimitriou, "Container Security: Issues, Challenges, and the Road Ahead," *IEEE Access*, vol. 7, pp. 52976–52996, 2019, doi: 10.1109/ACCESS.2019.2911732.
- [4] F. Minna et al., "Understanding the Security Implications of Kubernetes Networking," *IEEE Security & Privacy*, vol. 19, no. 5, pp. 46–56, Sep./Oct. 2021.
- [5] A. Sissodiya et al., "Formal Verification for Preventing Misconfigured Access Policies in Kubernetes Clusters," *IEEE Access*, vol. 13, 2025, doi: 10.1109/ACCESS.2025.3597504.
- [6] C. Cesarano and R. Natella, "KubeFence: Security Hardening of the Kubernetes Attack Surface," in *Proc. 55th IEEE/IFIP DSN*, 2025, doi: 10.1109/DSN64029.2025.00054.
- [7] M. Souppaya, J. Morello, and K. Scarfone, *Application Container Security Guide*, NIST SP 800-190, NIST, Sep. 2017, doi: 10.6028/NIST.SP.800-190.
- [8] Center for Internet Security, *CIS Kubernetes Benchmark*, CIS, 2024. [Online]. Available: <https://www.cisecurity.org/benchmark/kubernetes>
- [9] PCI Security Standards Council, *PCI DSS v4.0, PCI SSC*, Mar. 2022. [Online]. Available: <https://www.pcisecuritystandards.org>
- [10] L. Rice, *Container Security: Fundamental Technology Concepts that Protect Containerized Applications*, O'Reilly Media, 2020, ISBN: 978-1492056706.
- [11] Open Policy Agent Project, *OPA: Policy-based Control for Cloud Native Environments*, CNCF, 2021. [Online]. Available: <https://www.openpolicyagent.org>
- [12] A. Verdet, M. Hamdaqa, L. Da Silva, and F. Khomh, "Exploring Security Practices in Infrastructure as Code," *Empirical Softw. Eng.*, vol. 29, Art. 182, 2024, doi: 10.1007/s10664-024-10610-0.
- [13] M. R. Saleh Sedghpour, C. Klein, and J. Tordsson, "An Empirical Study of Service Mesh Traffic Management Policies for Microservices," in *Proc. ACM/SPEC ICPE*, 2022, doi: 10.1145/3489525.3511686.
- [14] D. A. Hahn, D. Davidson, and A. G. Bardas, "MisMesh: Security Issues and Challenges in Service Meshes," *SecureComm 2020*, LNICST vol. 335, Springer, 2020, pp. 140–151.
- [15] W. Li et al., "Service Mesh: Challenges, State of the Art, and Future Research Opportunities," in *Proc. IEEE SOSE 2019*, pp. 122–127.
- [16] R. Chandramouli and Z. Butcher, *Building Secure Microservices-based Applications Using Service-Mesh Architecture*, NIST SP 800-204A, NIST, May 2020, doi: 10.6028/NIST.SP.800-204A.
- [17] J. Elson and A. Cerpa, *Internet Content Adaptation Protocol (ICAP)*, IETF RFC 3507, Apr. 2003. Available: <https://datatracker.ietf.org/doc/html/rfc3507>
- [18] A. Y. Wong, E. G. Chekole, M. Ochoa, and J. Zhou, "On the Security of Containers: Threat Modeling, Attack Analysis, and Mitigation Strategies," *Computers & Security*, vol. 135, Art. 103490, Dec. 2023, doi: 10.1016/j.cose.2023.103490.
- [19] M. Hausenblas and S. Schimanski, *Programming Kubernetes: Developing Cloud-Native Applications*, O'Reilly Media, Jul. 2019, ISBN: 978-1492047100.
- [20] J. Dobies and J. Wood, *Kubernetes Operators: Automating the Container Orchestration Platform*, O'Reilly Media, Mar. 2020, ISBN: 978-1492048046.
- [21] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *Commun. ACM*, vol. 59, no. 5, pp. 50–57, May 2016, doi: 10.1145/2898442.2898444.
- [22] T.-T. Nguyen, Y.-J. Yeom, T. Kim, D.-H. Park, and S. Kim, "Horizontal Pod Autoscaling in Kubernetes for Elastic Container Orchestration," *Sensors*, vol. 20, no. 16, Art. 4621, Aug. 2020, doi: 10.3390/s20164621.
- [23] CNCF TAG Security, *Cloud Native Security Whitepaper*, Version 2, CNCF, May 2022. Available: <https://www.cncf.io/reports/cloud-native-security-whitepaper>
- [24] D. D'Silva and D. Ambawade, "Building A Zero Trust Architecture Using Kubernetes," in *Proc. IEEE I2CT*, Pune, Apr. 2021, pp. 1–8.
- [25] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, *Zero Trust Architecture*, NIST SP 800-207, NIST, Aug. 2020, doi: 10.6028/NIST.SP.800-207.